6.3 Double-Ended Queues

We next consider a queue-like data structure that supports insertion and deletion at both the front and the back of the queue. Such a structure is called a ***double-ended queue***, or ***deque***, which is usually pronounced "deck" to avoid confusion with the dequeue method of the regular queue ADT, which is pronounced like the abbreviation "D.Q."

The deque abstract data type is more general than both the stack and the queue ADTs. The extra generality can be useful in some applications. For example, we described a restaurant using a queue to maintain a waitlist. Occassionally, the first person might be removed from the queue only to find that a table was not available; typically, the restaurant will re-insert the person at the *first* position in the queue. It may also be that a customer at the end of the queue may grow impatient and leave the restaurant. (We will need an even more general data structure if we want to model customers leaving the queue from other positions.)

6.3.1 The Deque Abstract Data Type

To provide a symmetrical abstraction, the deque ADT is defined so that deque D supports the following methods:

D.add_first(e): Add element e to the front of deque D.

D.add_last(e): Add element e to the back of deque D.

D.delete_first(): Remove and return the first element from deque D; an error occurs if the deque is empty.

D.delete_last(): Remove and return the last element from deque D; an error occurs if the deque is empty.

Additionally, the deque ADT will include the following accessors:

D.first(): Return (but do not remove) the first element of deque D; an error occurs if the deque is empty.

D.last(): Return (but do not remove) the last element of deque D; an error occurs if the deque is empty.

D.is_empty(): Return True if deque D does not contain any elements.

len(D): Return the number of elements in deque D; in Python, we implement this with the special method __len__.

Example 6.5: *The following table shows a series of operations and their effects on an initially empty deque D of integers.*

Operation	Return Value	Deque
D.add_last(5)	–	[5]
D.add_first(3)	–	[3, 5]
D.add_first(7)	–	[7, 3, 5]
D.first( )	7	[7, 3, 5]
D.delete_last( )	5	[7, 3]
len(D)	2	[7, 3]
D.delete_last( )	3	[7]
D.delete_last( )	7	[ ]
D.add_first(6)	–	[6]
D.last( )	6	[6]
D.add_first(8)	–	[8, 6]
D.is_empty( )	False	[8, 6]
D.last( )	6	[8, 6]

6.3.2 Implementing a Deque with a Circular Array

We can implement the deque ADT in much the same way as the ArrayQueue class provided in Code Fragments 6.6 and 6.7 of Section 6.2.2 (so much so that we leave the details of an ArrayDeque implementation to Exercise P-6.32). We recommend maintaining the same three instance variables: _data, _size, and _front. Whenever we need to know the index of the back of the deque, or the first available slot beyond the back of the deque, we use modular arithmetic for the computation. For example, our implementation of the last( ) method uses the index

```
back = (self._front + self._size - 1) % len(self._data)
```

Our implementation of the ArrayDeque.add_last method is essentially the same as that for ArrayQueue.enqueue, including the reliance on a _resize utility. Likewise, the implementation of the ArrayDeque.delete_first method is the same as ArrayQueue.dequeue. Implementations of add_first and delete_last use similar techniques. One subtlety is that a call to add_first may need to wrap around the beginning of the array, so we rely on modular arithmetic to circularly *decrement* the index, as

```
self._front = (self._front - 1) % len(self._data)          # cyclic shift
```

The efficiency of an ArrayDeque is similar to that of an ArrayQueue, with all operations having $O(1)$ running time, but with that bound being amortized for operations that may change the size of the underlying list.

6.3.3 Deques in the Python Collections Module

An implementation of a deque class is available in Python's standard collections module. A summary of the most commonly used behaviors of the collections.deque class is given in Table 6.4. It uses more asymmetric nomenclature than our ADT.

Our Deque ADT	**collections.deque**	Description
len(D)	len(D)	number of elements
D.add_first( )	D.appendleft( )	add to beginning
D.add_last( )	D.append( )	add to end
D.delete_first( )	D.popleft( )	remove from beginning
D.delete_last( )	D.pop( )	remove from end
D.first( )	D[0]	access first element
D.last( )	D[−1]	access last element
	D[j]	access arbitrary entry by index
	D[j] = val	modify arbitrary entry by index
	D.clear( )	clear all contents
	D.rotate(k)	circularly shift rightward k steps
	D.remove(e)	remove first matching element
	D.count(e)	count number of matches for e

Table 6.4: Comparison of our deque ADT and the collections.deque class.

The collections.deque interface was chosen to be consistent with established naming conventions of Python's list class, for which append and pop are presumed to act at the end of the list. Therefore, appendleft and popleft designate an operation at the beginning of the list. The library deque also mimics a list in that it is an indexed sequence, allowing arbitrary access or modification using the D[j] syntax.

The library deque constructor also supports an optional maxlen parameter to force a fixed-length deque. However, if a call to append at either end is invoked when the deque is full, it does not throw an error; instead, it causes one element to be dropped from the opposite side. That is, calling appendleft when the deque is full causes an implicit pop from the right side to make room for the new element.

The current Python distribution implements collections.deque with a hybrid approach that uses aspects of circular arrays, but organized into blocks that are themselves organized in a doubly linked list (a data structure that we will introduce in the next chapter). The deque class is formally documented to guarantee $O(1)$-time operations at either end, but $O(n)$-time worst-case operations when using index notation near the middle of the deque.